Prompt técnico utilizado

LENGUAJE DE PROGRAMACIÓN:

Python 3.11 o superior, solo biblioteca estándar (threading, queue, logging, dataclasses, enum, random, time).

No usar asyncio ni frameworks externos: la concurrencia debe resolverse con hilos y primitivas explícitas.

No usar números mágicos

métodos o funciones de no más de 15 líneas.

El código debe estar escrito en idioma inglés.

Uso de patrones de diseño.

Espacios en lugar de tabulaciones (2 espacios).

Generar un entorno docker para correr el programa. Detallar los comandos para ejecutarlo en el readme final.

Generar una interfaz html simple en donde pueda ver las operaciones que ocurren

PLATAFORMA:

Aplicación de consola multihilo que simule la operación de una aerolínea.

El tiempo de simulación debe ser acelerado y configurable (por ejemplo, 1 segundo real = 10 minutos simulados).

Los parámetros (cantidad de aviones, vuelos, asignadores, semilla aleatoria, duración) deben poder configurarse por línea de comandos o archivo de configuración.

FUNCIONALIDAD A IMPLEMENTAR:

Desarrollar un sistema que asigne aviones a vuelos de forma dinámica según necesidad y disponibilidad, mientras llegan nuevos vuelos, se completan otros y ocurren eventos imprevistos.

REQUISITOS ESPECÍFICOS:

Entidades:

Avión: id, modelo, capacidad de pasajeros, autonomía, aeropuerto actual, estado.

Vuelo: id, origen, destino, horario de salida, duración, pasajeros, prioridad (emergencia, VIP, regular, carga), estado.

Hilos obligatorios:

Un hilo generador de vuelos (demanda aleatoria).

Al menos 3 hilos asignadores que operen en simultáneo sobre el mismo estado compartido.

Un hilo por avión que simule el vuelo y el turnaround.

Un hilo de eventos externos (fallas, mantenimientos, cancelaciones y demoras aleatorias).

Un hilo monitor/auditor que reporte el estado de la flota y verifique invariantes sin detener el sistema.

Asignación: un avión es compatible con un vuelo si tiene capacidad y autonomía suficientes, está en el aeropuerto de origen y está disponible. Entre varios compatibles, elegir el que minimice tiempo ocioso o reposicionamiento.

Prioridades: los vuelos de mayor prioridad se atienden primero. Para evitar starvation, implementar aging (la prioridad efectiva aumenta con el tiempo de espera).

Eventos imprevistos:

Un avión puede pasar a mantenimiento mientras está asignado o en evaluación. En ese caso, el vuelo vuelve a pendiente y se reasigna.

Un vuelo demorado puede afectar al siguiente vuelo del mismo avión.

Sincronización (obligatorio justificarla en el README):

Lock por avión, y adquisición múltiple siempre en orden global por id para evitar deadlock.

Condition o Semaphore para que los vuelos pendientes esperen la liberación de aviones, sin espera activa.

Prohibido usar time.sleep como mecanismo de sincronización.

Prohibido el patrón check-then-act sin lock (no consultar disponibilidad y asignar en pasos separados).

El monitor debe leer un snapshot consistente del estado.

Máquina de estados:

Avión (mínimo 5 estados): DISPONIBLE, ASIGNADO, EN_VUELO, TURNAROUND, MANTENIMIENTO.

Vuelo (mínimo 5 estados): PENDIENTE, ASIGNADO, EN_CURSO, COMPLETADO, CANCELADO.

Las transiciones válidas deben estar definidas en una tabla y cualquier transición inválida debe lanzar excepción.

Invariantes (verificarlos con aserciones y en el monitor):

Un avión nunca está asignado a dos vuelos con horarios solapados.

Un vuelo tiene como máximo un avión.

Un avión en mantenimiento nunca está asignado ni en vuelo.

La cantidad de aviones por estado suma el total de la flota.

Ningún vuelo compatible espera indefinidamente si hay un avión adecuado disponible.

Logs: usar logging con nivel, timestamp, nombre del hilo e identificadores. Registrar cada cambio de estado (avión AV-03: DISPONIBLE -> ASIGNADO (vuelo

FL-120)), cada asignación, reasignación y cancelación, y las violaciones de invariantes como ERROR.

Apagado ordenado: al terminar la simulación o con Ctrl+C, todos los hilos deben finalizar sin quedar bloqueados (usar threading.Event y sentinelas en colas).

Reproducibilidad: toda aleatoriedad debe depender de una semilla configurable.

ENTREGABLES:

Estructura modular de archivos (models.py, scheduler.py, workers.py, monitor.py, main.py).

Un conjunto de tests de estrés (por ejemplo, 50 aviones, 500 vuelos, 8 asignadores) que verifiquen los invariantes al finalizar.

Un README.md que explique las decisiones de sincronización, cómo se previene deadlock y starvation, y cómo ejecutar el programa.

Una breve sección con los riesgos de concurrencia que el diseño evita y cómo se podrían reproducir si se quitara cada mecanismo.